

Special Topics in Discrete Mathematics: Advanced Topics in Cryptography

Theory Exercise 1

Panayi Andreas^{*}
MSc: A.L.MA.
Student ID: 711514240008

School of Electrical and Computer Engineering,
National Technical University of Athens
Athens, Greece

April 2026



National Technical
University of Athens

^{*}Writing was done using X_YL^AT_EX

Contents

Problem 1	1
Description	1
Solution	1
Problem 2	2
Description	2
Solution	2
Problem 3	4
Description	4
Solution	4
Problem 4	5
Description	5
Solution	5
Problem 5	7
Description	7
Solution	7

Problem 1

Description

Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ be a collision-resistant hash function and define $G(x) = H(H(x))$. Use a reduction to show that G is a collision-resistant hash function.

Solution

Assume, towards a contradiction, that G is not collision resistant. Then there exists a PPT adversary A that finds a collision for G with non-negligible probability.

We construct a PPT adversary B that finds a collision for H as shown by the reduction below.

Algorithm $B(1^\kappa)$:

```
( $x_1, x_2$ )  $\leftarrow A(1^\kappa)$ 
 $y_1 \leftarrow H(x_1), \quad y_2 \leftarrow H(x_2)$ 
if  $y_1 \neq y_2$  then return ( $y_1, y_2$ )
else return ( $x_1, x_2$ )
```

Correctness:

Clearly B is PPT, since it invokes A once and performs only a constant number of additional computations.

Now whenever A succeeds, we have $x_1 \neq x_2$ and $G(x_1) = G(x_2)$.

By the definition of G , this means $H(H(x_1)) = H(H(x_2))$.

Equivalently, if we let $y_1 = H(x_1)$ and $y_2 = H(x_2)$, then $H(y_1) = H(y_2)$.

If $y_1 \neq y_2$, then (y_1, y_2) yields a collision for H , so B succeeds.

If $y_1 = y_2$, then $H(x_1) = H(x_2)$, and since $x_1 \neq x_2$, the pair (x_1, x_2) yields a collision for H . Again B succeeds.

Therefore, whenever A finds a collision for G , the adversary B finds a collision for H . Hence B succeeds with non-negligible probability, contradicting the collision resistance of H .

Therefore G is collision resistant.

□

Problem 2

Description

Given a collision resistant hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$, create a hash function $G : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ with the following properties:

1. G is collision resistant.
2. The first two bits of G are predictable (a bit is predictable if it can be successfully “guessed” with probability $\frac{1}{2} + \text{non-negligible}$ when the input is chosen uniformly at random).

Write out explicitly what this G function is, then define and prove the above two properties formally. For the first property, use a reduction.

Solution

Assuming $\{H_\kappa : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa\}_\kappa$ represents a family of collision-resistant hash functions parameterized by the security parameter κ , we construct a hash family G_κ with the desired properties.

We define $G_\kappa : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ as:

$$G_\kappa(x) = 00 \parallel H_{\kappa-2}(x)$$

where $\parallel$ denotes string concatenation. The total output length is exactly $2 + (\kappa - 2) = \kappa$ bits, satisfying the length requirement.

Correctness:

Assume, towards a contradiction, that G_κ is not collision resistant. Then there exists a PPT adversary A that finds a collision for G_κ with non-negligible probability.

We construct a PPT adversary B that finds a collision for $H_{\kappa-2}$ as shown by the reduction below.

Algorithm $B(1^\kappa)$:

$(x_1, x_2) \leftarrow A(1^\kappa)$

return (x_1, x_2)

Clearly B is PPT, since it invokes A once and performs only a constant number of additional computations.

Now whenever A succeeds, we have $x_1 \neq x_2$ and $G_\kappa(x_1) = G_\kappa(x_2)$.

By the definition of G_κ , this means $00 \parallel H_{\kappa-2}(x_1) = 00 \parallel H_{\kappa-2}(x_2)$.

Since the prefixes are identical, the suffixes must match exactly to satisfy string equality. Therefore, it must hold that $H_{\kappa-2}(x_1) = H_{\kappa-2}(x_2)$.

Since $x_1 \neq x_2$, the pair (x_1, x_2) yields a valid collision for $H_{\kappa-2}$. Thus, B succeeds.

Therefore, whenever A finds a collision for G_κ , the adversary B finds a collision for $H_{\kappa-2}$. Hence B succeeds with non-negligible probability, contradicting the collision resistance of $H_{\kappa-2}$.

Therefore G_κ is collision resistant. □

Predictability:

By the definition of our construction, $G_\kappa(x) = 00 \parallel H_{\kappa-2}(x)$. The first two bits of the output are always fixed to 00, completely independent of the input x .

Consider a PPT adversary A' that always guesses 0 for the first bit. Regardless of the uniformly

random input x , the first bit is 0. The probability that A' correctly guesses the bit is exactly 1. We can express this success probability in the format required by the definition:

$$\Pr[A' \text{ is correct}] = 1 = \frac{1}{2} + \frac{1}{2}$$

The advantage over a random guess is $\nu(\kappa) = \frac{1}{2}$. Since a constant function of $\frac{1}{2}$ does not approach zero as κ grows, it is by definition non-negligible. The exact same logic applies to the second bit. Hence, the first two bits of G_κ can be predicted with probability $\frac{1}{2} + \text{non-negligible}$. □

Problem 3

Description

Construct a correct but insecure signature scheme. Write the full implementation of your signature scheme in pseudocode. Prove its correctness and insecurity.

Solution

We propose the "Leaky Key" signature scheme. Let $f : \{0, 1\}^\kappa \rightarrow \{0, 1\}^\kappa$ be a preimage-resistant (one-way) function (such as SHA-256). In this scheme, generating a signature for any message inadvertently leaks the entire secret key.

Algorithm $\text{Gen}(1^\kappa)$:

```
 $sk \leftarrow \{0, 1\}^\kappa$   
 $pk \leftarrow f(sk)$   
return  $(pk, sk)$ 
```

Algorithm $\text{Sig}(sk, m)$:

```
return  $sk$ 
```

Algorithm $\text{Ver}(pk, \sigma, m)$:

```
if  $f(\sigma) = pk$  then return True  
else return False
```

Proof of Correctness

According to the definition, a signature scheme is correct if for all messages m , verification succeeds when evaluated on a valid signature produced by the scheme. Let m be any message and let (pk, sk) be keys generated by $\text{Gen}(1^\kappa)$, so that $pk = f(sk)$. The signing function $\text{Sig}(sk, m)$ always outputs the signature $\sigma = sk$. The verification function $\text{Ver}(pk, m, \sigma)$ checks whether $f(\sigma) = pk$. Substituting our values, this checks if $f(sk) = f(sk)$, which is a tautology. Thus, the scheme always returns True.

Proof of Insecurity

To prove insecurity, we demonstrate a Polynomial-Time Adversary $\mathcal{A}$ that can win the existential forgery game with probability 1. Let the adversary play the game by interacting with the signing oracle $\mathcal{O}(\cdot)$ as follows:

Algorithm $\mathcal{A}(pk)$:

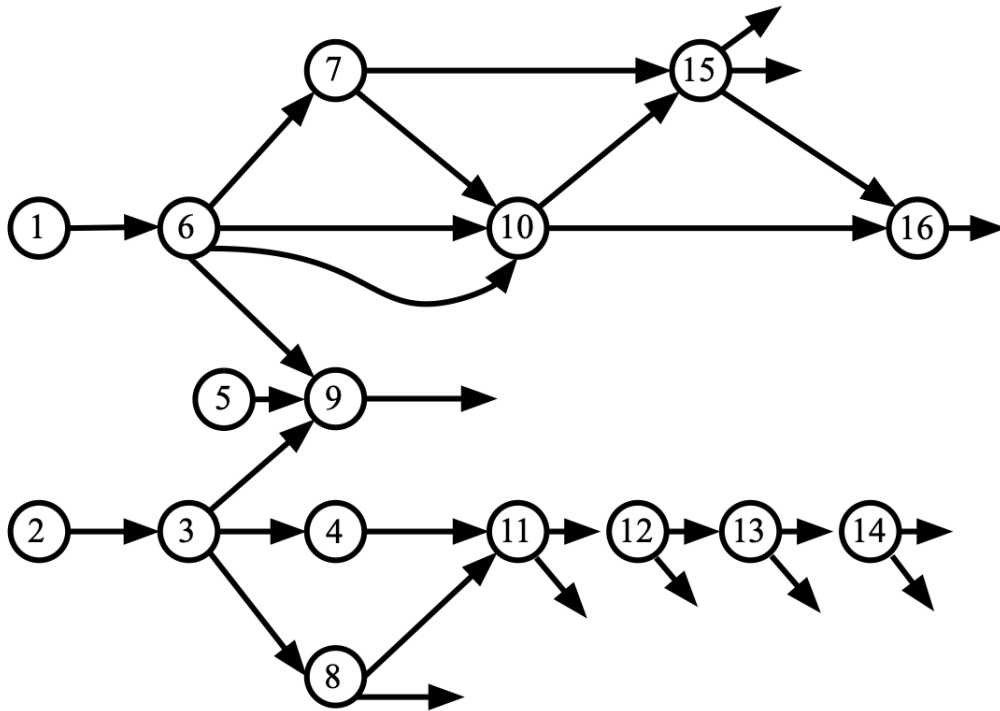
```
 $m_1 \leftarrow \text{"hello"}$   
 $\sigma_1 \leftarrow \mathcal{O}(m_1)$   
 $m^* \leftarrow \text{"attack"}$   
 $\sigma^* \leftarrow \sigma_1$   
return  $(m^*, \sigma^*)$ 
```

By construction of the scheme, querying the oracle on m_1 returns $\sigma_1 = sk$. Thus, the adversary successfully intercepts the secret key. The adversary then outputs a fresh message $m^* \notin M$ (since $m^* \neq m_1$) along with the forgery $\sigma^* = sk$. Then, the game checks if $\text{Ver}(pk, \sigma^*, m^*) = 1$ and $m^* \notin M$. Since $f(\sigma^*) = f(sk) = pk$, verification returns True. Since m^* was never queried to the oracle, the second condition is also met. The game returns True, proving the scheme is insecure. $\square$

Problem 4

Description

Consider the complete transaction graph **accepted and stored in the database of an honest node** illustrated by the following graph. The circles represent transactions and the numbers within them are the txids. The outgoing edges are outputs, and the incoming edges are inputs. For each transaction, edges appear ordered from top to bottom. For this graph:



1. Identify the coinbase transactions.
2. Identify the double spending transactions. For each double spending transaction, identify the transaction it is conflicting with.
3. Identify the UTXO set.
4. Identify the transactions for which the Weak Conservation Law does not hold.
5. Identify the outputs of transactions that are partially, but not fully, spent.

To identify transactions, use their txids. To identify an output, use outpoint notation. Note that transaction 6 has four outputs.

Solution

1. **Coinbase transactions.** By definition, coinbase transactions have no inputs. From the graph, these are
 $\{1, 2, 5\}$.
2. **Double spending transactions.** There are no double spending transactions. Indeed, the graph is stated to be *accepted and stored in the database of an honest node*; such a node validates transactions before accepting them, so no conflicting transactions would be present in its database.

3. **UTXO set.** The unspent transaction outputs are exactly the outputs that are not consumed by any outgoing edge in the graph. Hence the UTXO set is

$$\{(15, 0), (15, 1), (16, 0), (9, 0), (14, 0), (13, 1), (12, 1), (11, 1), (8, 1)\}.$$

4. **Transactions for which the Weak Conservation Law does not hold.** There are no such transactions among the non-coinbase transactions. Since the graph is accepted and stored by an honest node, all transactions in its database have already been validated, and thus satisfy the Weak Conservation Law. Coinbase transactions are excluded from this requirement by definition.
5. **Outputs of transactions that are partially, but not fully, spent.** There are no such outputs. In the UTXO model, an output is spent as a whole; partial spending of a single output is not allowed. Therefore, every output is either fully spent or completely unspent.

Problem 5

Description

The honest proof-of-work algorithm begins by sampling a uniformly random κ -bit nonce, then iterates for a polynomial number of repetitions while incrementing the nonce. Prove that, if a constant number of honest parties execute this algorithm a polynomial number of times, the probability that they query the hash using the same nonce is negligible.

Solution

Let c be the number of honest parties, where $c = O(1)$. Assume each honest party executes the proof-of-work algorithm at most $r(\kappa)$ times, where r is a polynomial, and that each execution makes at most $q(\kappa)$ hash queries, where q is also a polynomial. By the algorithm, each execution first samples a starting nonce uniformly at random from $\{0, 1\}^\kappa$ and then queries the hash on consecutive nonce values obtained by incrementing this starting nonce.

Hence the total number of executions is at most $M(\kappa) = c r(\kappa)$.

Fix two different executions. Let their randomly chosen starting nonces be $S, S' \in \{0, \dots, 2^\kappa - 1\}$, chosen independently and uniformly. The first execution queries the hash on the nonces

$$S, S + 1, \dots, S + q(\kappa) - 1,$$

and the second on

$$S', S' + 1, \dots, S' + q(\kappa) - 1.$$

These two executions use a common nonce if and only if there exist integers $a, b \in \{0, \dots, q(\kappa) - 1\}$ such that

$$S + a = S' + b.$$

Equivalently,

$$S - S' = b - a.$$

Since $b - a$ must lie in the interval

$$\{-(q(\kappa) - 1), \dots, q(\kappa) - 1\},$$

for a fixed value of S' there are at most $2q(\kappa) - 1$ values of S for which the two executions query a common nonce. Therefore

$$\Pr[\text{these two executions query a common nonce}] \leq \frac{2q(\kappa) - 1}{2^\kappa} \leq \frac{2q(\kappa)}{2^\kappa}.$$

Now apply the union bound over all pairs of executions. There are at most $\binom{M(\kappa)}{2}$ such pairs, so

$$\begin{aligned} \Pr[\exists \text{ two honest executions that query a common nonce}] &\leq \binom{M(\kappa)}{2} \cdot \frac{2q(\kappa)}{2^\kappa}. \\ \Leftrightarrow \Pr[\exists \text{ two honest hash queries using the same nonce}] &\leq \binom{c r(\kappa)}{2} \cdot \frac{2q(\kappa)}{2^\kappa}. \end{aligned}$$

Since c is constant and $r(\kappa), q(\kappa)$ are polynomials, the right-hand side is of the form $\frac{\text{poly}(\kappa)}{2^\kappa}$, which is negligible in κ .

Hence the probability that honest parties query the hash using the same nonce is negligible, as required.